

Bazele Programării pe Obiecte

C 3-5

Elemente ale limbajului VB



Elemente de bază

- *Limbajul nu este case sensitive*
- *Setul de caractere: litere (A-Z, a-z), cifre (0-9), caractere speciale (+ - / \ ^ () [] { } . , ; : _ ! # % & \$ etc.), perechi de caractere (<=, >=, <>, =, <, >, +=, -=, *=, etc.), separatori (space, Tab, Enter).*
- *Este recomandat ca fiecare instrucțiune să fie scrisă pe o linie separată;*
 - *_ permite împărțirea unei instrucțiuni pe mai multe linii;*
 - *: permite separarea a două instrucțiuni pe aceeași linie.*
- *Comentariile se pot include folosind ' (apostrof)*

Identificatori

- *Un **identificator** este numele dat unui element dintr-un program (variabilă, constantă, tip definit de utilizator, enumerare, funcție, subrutină, obiect, metodă, proprietate, control, form, modul, etc.)*
- *Regulile de definire a unui identificator:*
 - *conține numai litere, cifre și `_`, și nu poate începe cu o cifră*
 - *nu conține spații sau alte caractere speciale : , . % ! \$ & #...*
 - *nu poate fi un cuvânt cheie (Dim, Private, Public, Array, For, Next, If, Then, Else, End, Exit etc.)*
 - *lungime maximă: variabile, constante – 255 caractere; controale, module, clase, form – 40 caractere*
 - *trebuie să fie unic (nu pot fi doi identificatori identici)*

Tipuri de date (1)

- *Tipul de date determină ce fel de valori pot fi stocate într-o variabilă / constantă / proprietate, cum sunt stocate aceste date, precum și setul de operații permise asupra acestora.*
- *Numerice:*
 - **Byte, Short, Integer, Long** – pentru numere întregi:
 - **Single, Double, Decimal** – pentru numere reale:
- *Text:*
 - **String** – pentru șiruri de caractere
 - **Char** – pentru un caracter
- *Alte tipuri de date:*
 - **Boolean** – pentru valori logice (true / false)
 - **Date** – pentru dată și oră
 - **Object** – poate fi utilizat pentru a stoca date de orice tip

Tipuri de date (2)

Tip date	Interval		Bytes
Byte	0 ... 255		1
Short	-32,768 ... 32,767		2
Integer	-2,147,483,648 ... 2,147,483,647		4
Long	-9,223,372,036,854,775,808 ... 9,223,372,036,854,775,807		8
Single	1.4E-45 ... 3.4E+38		4
Double	4.9E-324 ... 1.7E+308		8
Decimal	1.0E-28 ... 7.9E+28		16
Date	January 1, 0001 ... December 31, 9999, 0:00:00 to 23:59:59		8
Boolean	True / False		2
Char	un caracter Unicode		2
String	0 ... ~2 miliarde (2^{31}) caractere Unicode	2 per caracter	
Object	date de orice tip		-

Variable și constante (1)

- O **variabilă** este folosită pentru stocarea datelor a căror valoare se poate modifica.
- Variabilele trebuie declarate înainte de utilizare specificând:
 - numele – identificatorul folosit pentru referirea variabilei
 - tipul – specifică tipul datelor ce pot fi stocate în variabilă
- Declararea unei variabile:
Dim varName **As** varType
- Atribuirea unei variabile:
varName = expr
- O **constantă** este similară cu o variabilă, cu excepția faptului că valoarea ei nu poate fi modificată.
Const constName **As** constType = constValue

Variable și constante (2)

- *O variabilă are două caracteristici importante:*
 - vizibilitatea – zona unde se poate folosi variabila
 - durata de viață – timpul în care variabilei îi este alocat spațiu în memorie
- *Vizibilitatea și durata de viață sunt controlate de locul și de modul în care este declarată variabila.*
- *Variabilele pot fi declarate:*
 - local – în interiorul unei subrutine
 - global – la nivel de clasă / form / modul
- *Vizibilitatea și durata de viață ale unei constante pot fi gestionate într-o manieră similară.*

Variable și constante (3)

- **Public** – pentru declararea variabilelor globale ce pot fi accesate din orice altă clasă / modul.

Public varName **As** varType

- **Private** – pentru declararea variabilelor globale ce pot fi accesate doar din clasa / modulul în care au fost declarate.

Private varName **As** varType

- **Dim** – pentru declararea variabilelor globale (clasă / modul) sau locale (subrutină).

Dim varName **As** varType

- **Static** – pentru declararea variabilelor locale care rămân alocate și după execuția blocului în care au fost declarate.

Static varName **As** varType

Declarații variabile / constante

- ***Exemple:***

Public intAge **As Integer**

Public strName **As String** = "student"

Private strAdress **As String**

Dim intA **As Integer**

Dim dblB, dblC **As Double**

Dim strS1 **As String**, datD1 **As Date**

Dim objM, objN ‘ dacă este omis, implicit tipul va fi Object

Dim intYear **As Short** = 2022

Static lstMyList **As ListBox**

Static strS2 **As String** = "abcde", strS3 **As String**

Static datD2 **As Date** = #1/9/2000#

Const cintGrade **As Integer** = 7

Const cstrName **As String** = "Ionescu"

Operatori (1)

- *Operatori aritmetici:*
 - $\wedge$ – ridicare la putere (ex: $a \wedge b$)
 - $*$ – înmulțire (ex: $a * b$)
 - $/$ – împărțire (ex: a / b)
 - $\backslash$ – împărțire întreagă (câtul împărțiri) (ex: $a \backslash b$)
 - **Mod** – modulo (restul unei împărțiri întregi) (ex: $a \text{ Mod } b$)
 - $+$ – adunare (ex: $a + b$)
 - $-$ – scădere (ex: $a - b$)
- *Reguli de precedență:*
 - mai întâi sunt evaluate expresiile din paranteze (),
 - ridicarea la putere înaintea altor operații aritmetice,
 - înmulțirea și împărțirea,
 - și la final adunarea și scăderea.

Operatori (2)

- *Operatori relaționali:*
 - **<, >, <=, >=, =, <>** – comparare numerică și alfabetică
 - **Like** – compară un string cu un pattern
 - **Is** – determină dacă două obiecte fac referire la aceeași instanță
- *Operatori de concatenare:*
 - **+, &** – concatenarea a două stringuri
- *Operatori logici :*
 - **Not** – negare logică
 - **And, AndAlso** – și logic (ambele expresii trebuie să fie adevărate)
 - **Or, OrElse** – sau logic (una sau ambele expresii trebuie să fie adevărate)
 - **Xor** – sau exclusiv (una din cele două expresii trebuie să fie adevărată, dar nu amândouă)

Instrucțiuni (1)

- *Tipuri de instrucțiuni:*
 - declarative
 - de atribuire
 - executabile
- *Instrucțiunile declarative sunt folosite pentru a declara variabile, constante, masive, subrutine, funcții, proprietăți etc.*
- *Instrucțiuni de atribuire:*
 - atribuire simplă: `varName = expr`
 - compusă (atribuire compusa cu operatorii aritmetici sau de concatenare: `^=, *=, /=, \=, +=, -=, &=`): `varName op= expr`
- *O instrucțiune executabilă efectuează o acțiune – apel subrutină, instrucțiuni alternative / repetitive, tratare excepții etc*

Instrucțiuni (2)

- *VB are instrucțiuni pentru implementarea structurilor de control fundamentale:*
 - *alternative: If...Then...Else; Select Case*
 - *repetitive: For...Next; For Each...Next; Do { While | Until }...Loop; Do ...Loop { While | Until }*
- *Precum și alte instrucțiuni executabile pentru a efectua diferite acțiuni precum:*
 - *apelul unei metode sau subrutine;*
 - *salt la altă linie de cod;*
 - *tratarea excepțiilor;*
 - *salt peste iterații, ieșirea din instrucțiuni repetitive;*
 - *terminarea execuției unei subrutine, etc.*

If...Then...Else (1)

- *Sintaxa generală:*

If condition **Then**

[statements]

[**Elseif** elseif_condition **Then**

[elseif_statements]]

[**Else**

[else_statements]]

End If

- *Sintaxa single-line:*

If condition **Then** [statements] [**Else** [else_statements]]

If...Then...Else (2)

Exemple:

- *multi-line:*

If count = 0 **Then**

 mesaj = "lista nu are elemente."

Elseif count = 1 **Then**

 message = "lista are un element."

Else

 mesaj = "lista contine " & count & " elemente."

End If

- *single-line :*

If A > B **Then** D=A-B : A=-A **Else** D=B-A

If nota >=5 **Then** MessageBox.Show ("ai trecut")

If...Then...Else (3)

Structuri IF imbricate

- *Permit includerea structurilor IF în alte structuri IF (sau de alte tipuri):*

```
If condition_1 Then  
    statements1_true  
    If condition_2 Then  
        statements2_true  
    Else  
        statements2_false  
    End If
```

```
Else  
    statements1_false  
    If condition_3 Then  
        statements3_true  
    Else  
        statements3_false  
    End If  
End If
```

If...Then...Else (4)

- *Exemplu:*

If nota >=5 **Then**

 mesaj = " Ai promovat examenul "

If grade >=8 **Then**

 mesaj = mesaj + " cu o nota buna "

Else

 mesaj = mesaj + " cu o nota acceptabila "

End If

Else

 mesaj = " ai picat examenul "

End If

Select...Case (1)

- *Sintaxă generală:*

```
Select Case test_expression  
  [ Case expression_list_1  
    [ statements_1 ] ]  
  [ Case expression_list_2  
    [ statements_2 ] ]  
  .....  
  [ Case Else  
    [ else_statements ] ]  
End Select
```

Select...Case (3)

- *test_expression* este expresia care trebuie evaluată; trebuie să fie de tip elementar (numeric, string, boolean sau date)
- Expresiile *expression_list* conțin posibile valori pentru expresia de test; pot fi de orice tip de date, cu condiția să fie implicit convertibile la tipul expresiei *test_expression*. Listele pot conține valori și expresii multiple separate prin , (virgulă).
- Fiecare expresie din liste poate avea forma:
 - *expression_1 To expression_2* – specifică un interval de valori posibile pentru *test_expression*.
 - **Is** *comparison_operator expression* – specifică o relație (=, <>, <, <=, >, >=) pentru valorile posibile ale *test_expression*.
 - *expression* – specifică o singură valoare posibilă.

Select...Case (3)

Exemple:

- *exemplu 1:*

Select Case strChar

Case "0" To "9"

CharType = "cifra"

Case "A", "E", "I", "O", "U"

CharType = "vocala"

Case "A" To "Z"

CharType = "consoana"

Case Else

CharType = "alt caracter"

End Select

Select...Case (4)

- *exemplu 2:*

Select Case etaj

Case 1, 2

mesaj = "foloseste scarile"

Case 3 To 5

mesaj = "merge si pe scari, dar mai bine ia liftul"

Case 6 To 10

mesaj = "ia liftul"

Case Is >= 0

mesaj = "cladirea nu are atatea etaje"

Case Else

mesaj = "incorect"

End Select

For...Next (1)

- *Sintaxă generală:*

```
For counter = start_val To end_val [ Step step_val ]  
    [ statements ]  
    [ Continue For ]  
    [ statements ]  
    [ Exit For ]  
    [ statements ]  
Next [ counter ]
```

- **Continue For** – *trece la următoarea iterație (optional)*
- **Exit For** – *termină execuția buclei For (optional)*
- *For crescător* step_val > 0; *For descrescător* step_val < 0;
implicit (dacă step_val este omis) este 1 (valori consecutive)

For...Next (2)

Exemple:

- *exemplu 1:*

For index = 1 **To** 5

output = output + index.ToString () & " "

Next index ' output: 1 2 3 4 5

- *exemplu 2:*

For numar = 2 **To** 0 **Step** -0.25

output = output + numar.ToString () & " "

Next ' output: 2 1.75 1.5 1.25 1 0.75 0.5 0.25 0

- *exemplu 3:*

For i = ListBox1.Items.Count - 1 **To** 0 **Step** -1

ListBox1.Items.RemoveAt (i)

Next i ' sterge toate elementele unui listbox

For...Next (3)

- *exemplu 4 – Continue For și Exit For :*

For index = 1 **To** 100

If index >= 5 **And** index <= 8 **Then Continue For**

output = output + index.ToString () & " "

If index = 10 **Then Exit For**

Next

' output: 1 2 3 4 9 10

- *exemplu 5 – structuri For imbricate:*

For i = 1 **To** 3

For j = 1 **To** 4

ij = i.ToString () & " " & j.ToString () & vbCrLf

Text1.AppendText (ij)

Next j

Next i

For Each...Next (1)

- ***For Each...Next** repeta o prelucrare pentru fiecare element dintr-o colecție (grup) sau un masiv*
- *Sintaxă generală:*

For Each element **In** group

[statements]

[**Continue For**]

[statements]

[**Exit For**]

[statements]

Next [element]

- element – variabila utilizată pentru a itera prin elementele colecției
- group – *colecția prin care se iterează.*

For Each...Next (2)

- *Execuție:*
 - înainte de începerea buclei se evaluează colecția; colecția este evaluată o singură dată.
 - elementele din colecție sunt atribuite succesiv variabilei `element` și se efectuează prelucrările pentru fiecare element în parte.
 - când toate elementele din colecție (grup) au fost prelucrate (alocate succesiv variabilei `element`), bucla *For Each* se oprește.
 - este de evitat modificarea valorii variabilei `element` în interiorul buclei; modificarea variabilei `group` nu afectează colecția sau elementele acesteia, acesta fiind stabilită la intrarea în buclă.
- *For Each* se utilizează pentru iterarea prin toate elementele unei colecții care nu se dorește a fi modificată. Dacă colecția trebuie parcursă într-o anumită ordine sau se dorește modificarea elementelor colecției, se pot folosi alte instrucțiuni repetitive.

For Each...Next (3)

Exemple:

- *exemplu 1:*

```
Dim numberSeq() As Integer = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12}
For Each number In numberSeq
    If number <= 5 Then Continue For
    output += number.ToString ( ) & " "
Next number
```

' output: 6 7 8 9 10 11 12

- *exemplu 2:*

```
Dim thisControl As Windows.Forms.Control
For Each thisControl In Me.Controls
    thisControl.BackColor = Drawing.Color.Red
Next thisControl
```

' iterează prin toate controalele formului și le schimbă culoarea

Do { While | Until }...Loop (1)

- *Sintaxă Do While...Loop:*

Do While condition

[statements]

[**Continue Do**]

[**Exit Do**]

[statements]

Loop

- *Sintaxă Do Until...Loop:*

Do Until condition

[statements]

[**Continue Do**]

[**Exit Do**]

[statements]

Loop

Do { While | Until }...Loop (2)

Exemple:

- *exemplu 1:*

```
Dim index As Integer = 0
```

```
Do While index <= 10
```

```
    output = output + index.ToString ( ) & " "
```

```
    index += 1
```

```
Loop
```

```
' output: 0 1 2 3 4 5 6 7 8 9 10
```

sau:

```
Dim index As Integer = 0
```

```
Do Until index > 10
```

```
    output = output + index.ToString ( ) & " "
```

```
    index += 1
```

```
Loop
```

```
' output: 0 1 2 3 4 5 6 7 8 9 10
```

Do...Loop { While | Until } (1)

- *Sintaxa Do...Loop While:*

Do

[statements]

[**Exit Do**]

[**Continue Do**]

[statements]

Loop While condition

- *Sintaxa Do...Loop Until:*

Do

[statements]

[**Exit Do**]

[**Continue Do**]

[statements]

Loop Until condition

Do...Loop { While | Until } (2)

Exemple:

- *exemplu 1:*

```
Dim strNum As String
```

```
Do
```

```
    strNum = InputBox (" introdu un numar ")
```

```
Loop Until IsNumeric (strNum)
```

' repeta citirea până când este furnizată o valoare numerică

sau:

```
Dim strNum As String
```

```
Do
```

```
    strNum = InputBox (" introdu un numar ")
```

```
Loop While Not IsNumeric (strNum)
```

Gestionarea excepțiilor (1)

- *Instrucțiunile On Error implementează rutine de tratare a erorilor și specifică modul în care acestea sunt gestionate.*

On Error Resume Next

On Error GoTo line

- *Exemplu sintaxa:*

```
Public Sub MySub ()
```

```
    On Error GoTo ErrorHandler
```

```
    ' codul care poate genera eroare
```

```
    Exit Sub
```

```
ErrorHandler:
```

```
    ' codul care gestioneaza eroarea
```

```
    Resume Next ' reia executia cu linia urmatoare celei cu eroarea
```

```
End Sub
```

Gestionarea excepțiilor (2)

Exemple:

Dim x As Integer, s As String

- *exemplu 1:*

On Error Resume Next ' asigura gestionarea erorilor

z = Convert.ToInt32(s) ' conversie care poate genera eroare

TextBox1.Text = z.ToString()

- *exemplu 2:*

On Error GoTo myerror ' asigura gestionarea erorilor

z = Convert.ToInt32(s) ' conversie care poate genera eroare

TextBox1.Text = z.ToString()

Exit Sub

myerror: MessageBox.Show ("Eroare conversie")

Resume Next

Try...Catch...Finally (1)

- *Oferă mecanisme mai eficiente de gestionare a excepțiilor.*
- *Sintaxă generală:*

Try

[tryStatements]

[**Exit Try**]

[**Catch** [exception **As** exceptionType]

[catchStatements]

[**Exit Try**]]

[**Finally**

[finallyStatements]]

End Try

Try...Catch...Finally (2)

- *Execuție:*

- *Blocul Try conține codul care poate genera excepții. În cazul în care nu apare nicio excepție la execuție, instrucțiunile blocului Try se execută integral*
- *Blocul Try este urmat de una sau mai multe clauze Catch, care gestionează diferite excepții. Când apare o eroare într-un bloc Try, se caută blocul Catch care gestionează acea excepție specifică. Dacă nu se găsește, se caută blocul Catch care tratează excepțiile generale (de clasă Exception). Dacă nu este găsit niciun astfel de bloc Catch, execuția se oprește (excepție netratată).*
- *Blocul Finally se execută indiferent de cum decurge execuția blocului Try .*

Try...Catch...Finally (3)

Try

```
x = Convert.ToInt32(TextBox1.Text) : y = Convert.ToInt32(TextBox2.Text)
If boolInt = True Then z = x \ y Else z = x / y
rez = z.ToString
```

Catch ex As FormatException

```
MessageBox.Show(" introdu numere " + ex.Message)
```

Catch ex As DivideByZeroException

```
MessageBox.Show(" impartire la zero " + ex.Message)
```

Catch ex As ArithmeticException

```
MessageBox.Show(" exceptie aritmetica " + ex.Message)
```

Catch ex As Exception

```
MessageBox.Show(" a aparut o eroare " + ex.Message)
```

Finally

```
TextBox1.Text = rez
```

End Try



END